

Operating Systems

Rowan University

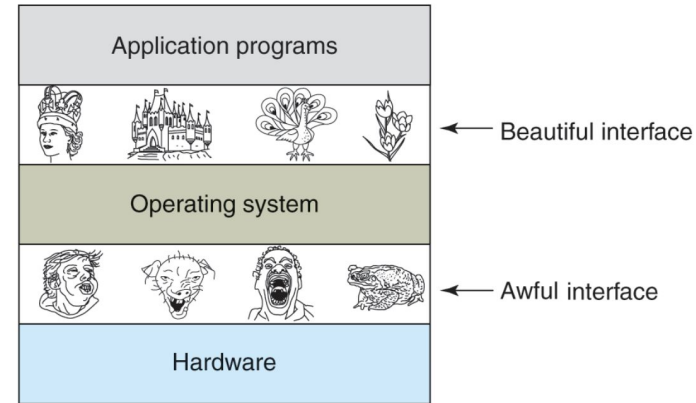
Overview

1. Syllabus Review
2. Computer Hardware Review

What is an Operating System?

- Two essential functions:
 - Provide clean abstract resources for application programmers
 - Manage hardware resources
- Architecture for most computers is difficult to work with
 - Primitive (binary) and awkward
- Operating system provide abstractions to help manage complexity

Figure 1-2

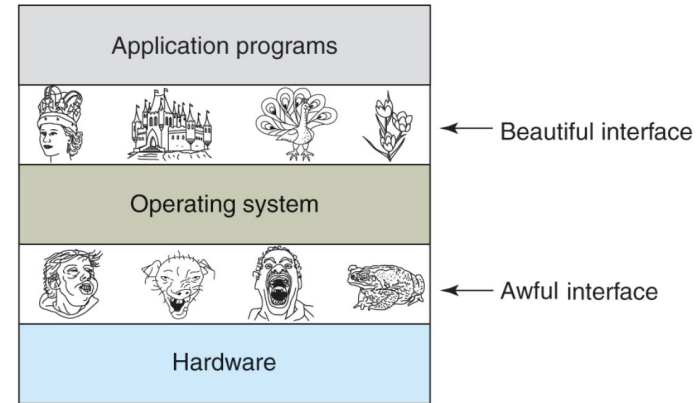


Operating systems turn awful hardware into beautiful abstractions.

What is an Operating System?

- Good abstractions:
 - Define and implement
 - Solve problems using the created abstractions.
 - Example: File abstraction for easier handling
- The Role of Operating Systems
 - Significance: Hide hardware complexities
 - Transformation: Turn hardware into clean, elegant abstractions
 - Real customers: Application programs
 - Objective: Provide consistent, elegant, and efficient abstractions.
- User Interface vs. Application Programs
 - Operating system's customers
 - Application programs
 - End users deal with user interfaces
 - Example: Windows desktop vs. command prompt

Figure 1-2



Operating systems turn awful hardware into beautiful abstractions.

Operating System as a Resource Manager

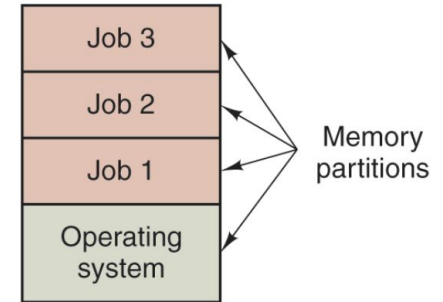
Managing Concurrent Programs

- Challenge: Multiple programs running simultaneously
- Example: Printing output simultaneously on the same printer
- Solution: Operating system buffers output, ensuring orderly printing

Resource Management in Multi-User Environments

- Importance: Prevent interference between users
- Shared Resources: Memory, I/O devices, information (files, databases)
- Operating System Tasks: Track resource usage, grant resource requests, mediate conflicts

Figure 1-5



A multiprogramming system with three jobs in memory.

Multiplexing

Types of Resource Multiplexing

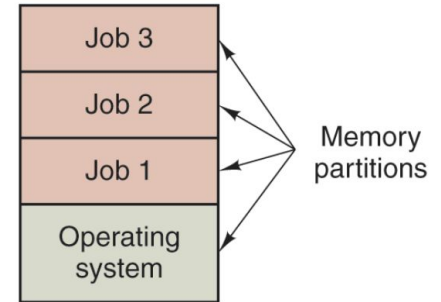
Time Multiplexing:

- Definition: Programs or users take turns using a resource
- Example: CPU allocation to different programs over time

Space Multiplexing:

- Definition: Each customer gets part of the resource
- Example: Main memory divided among running programs

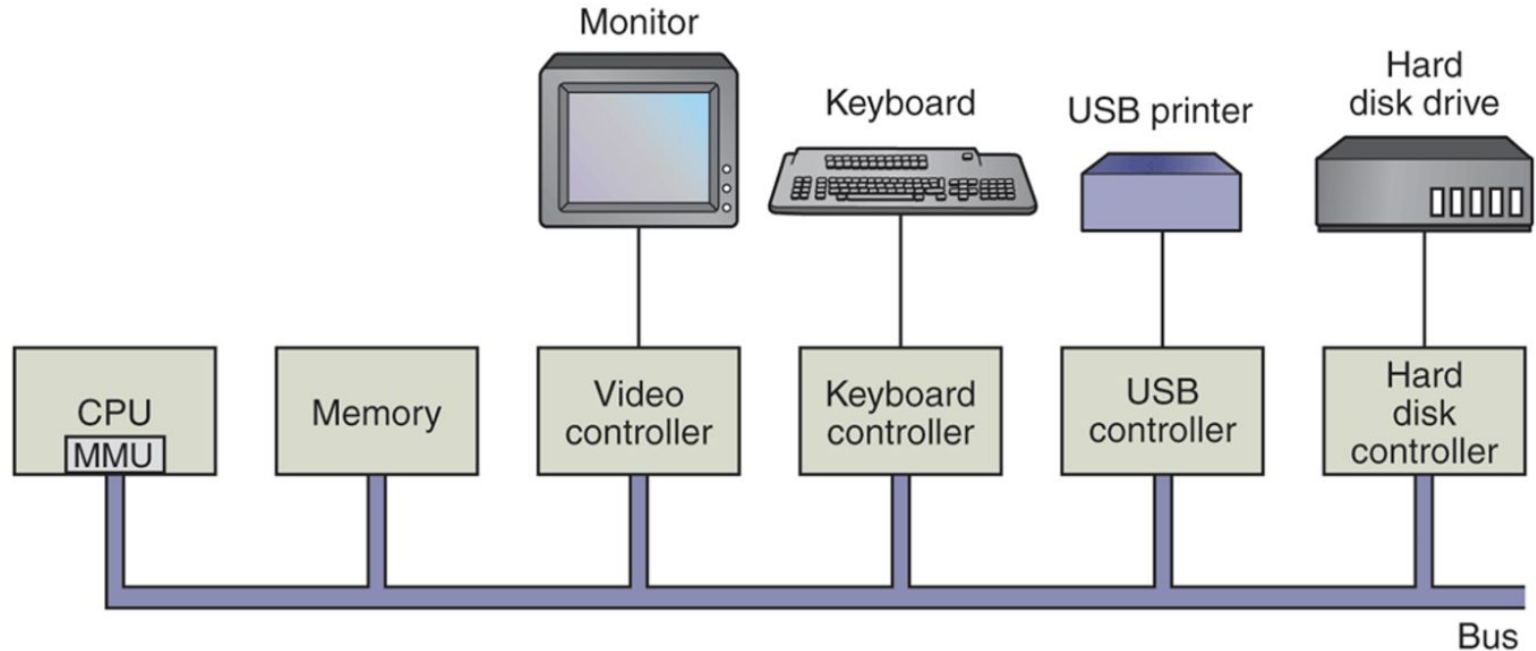
Figure 1-5



A multiprogramming system with three jobs in memory.

Computer Hardware Review

Figure 1-6

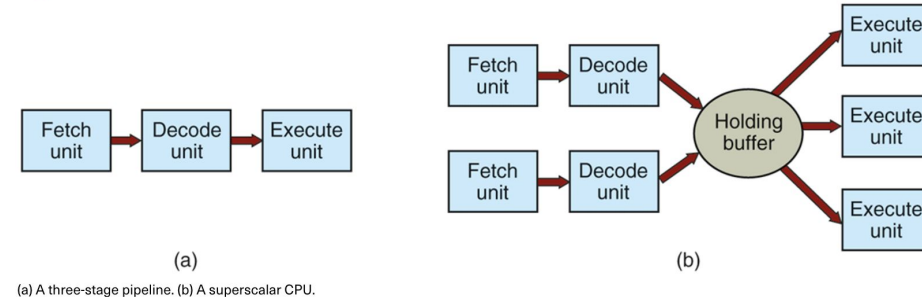


Some of the components of a simple personal computer.

Processors (CPUs)

- Fetches instructions from memory and executes them
- Each CPU has a specific set of instructions (e.g. x86 architecture)
- CPUs contain registers to hold key variables and temporary results
 - Program Counter: Holds the memory address of the next instruction
 - Stack Pointer: Points to the top of the current stack in memory
 - PSW (Program Status Word): Contains condition code bits, CPU priority, mode, and control bits
- Mode Operation:
 - Most CPUs have kernel mode and user mode
 - Kernel mode: Full hardware access; operating system runs here
 - User mode: Restricted instructions and features

Figure 1-7



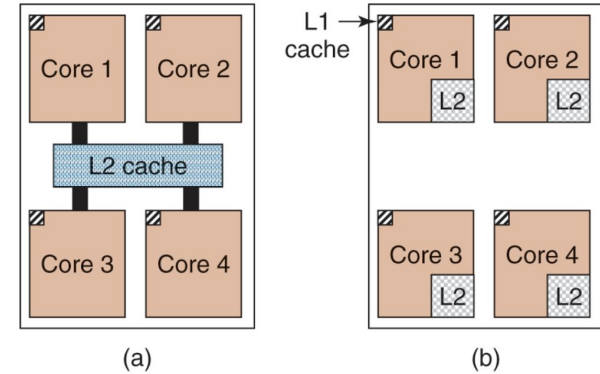
Operating System Awareness:

- Operating system must be aware of all registers
- During time multiplexing, the OS saves and restores registers when switching programs

Processors (CPUs)

- Multithreading
 - Allows CPU to hold state of two threads and switch between them on a nanosecond timescale
 - Multithreading does not offer true parallelism but reduces thread-switching time
- Implications for Operating System
 - Each thread appears as a separate CPU to the operating system
 - Inefficient scheduling may occur if the OS treats threads as separate CPUs
- Multicore Chips
 - Beyond multithreading, chips have multiple complete processors or cores
 - Multicore chips may have four, eight, or more cores
 - Each core has its own independent CPU

Figure 1-8

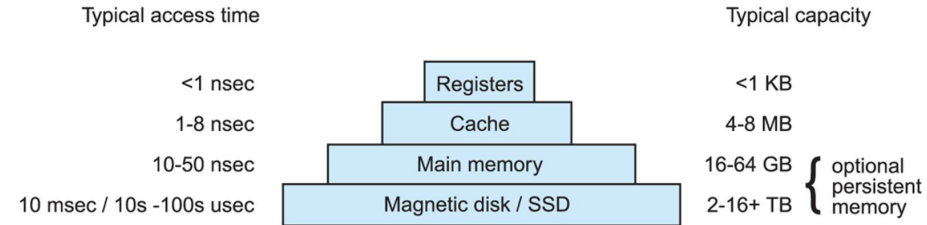


(a) A quad-core chip with a shared L2 cache. (b) A quad-core chip with separate L2 caches.

Main Memory

- Memory constructed as a hierarchy of layers for optimal performance
 - Each layer has varying speed, capacity, and cost per bit
- Registers are internal to the CPU, made of the same material, and as fast as the CPU
- Cache: Divided into cache lines, with the most used lines in a high-speed cache close to the CPU
- Main Memory
 - Workhorse of the memory system, usually referred to as RAM (Random Access Memory)
 - Main memory serves as a bridge between cache and nonvolatile memory

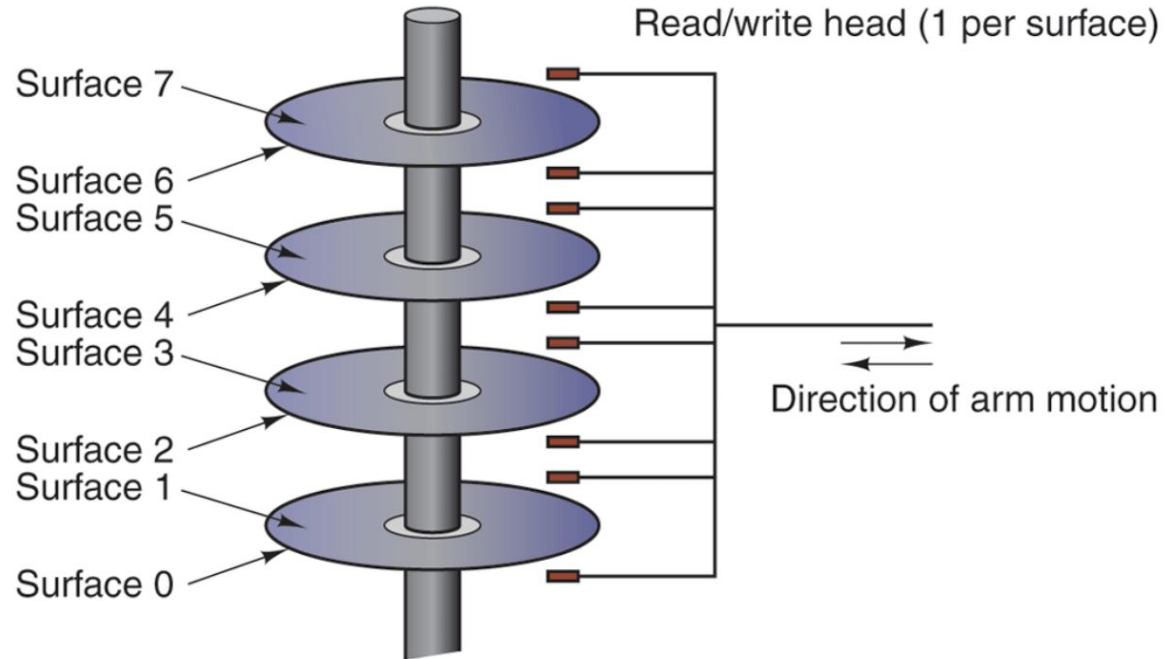
Figure 1-9



A typical memory hierarchy. The numbers are very rough approximations.

Storage

Figure 1-10

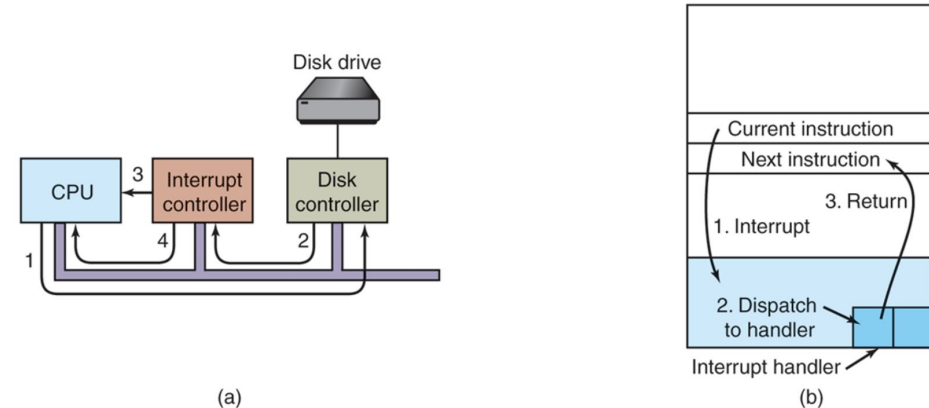


Structure of a disk drive.

Input/Output (I/O) Devices

- I/O devices generally consist of two parts: a controller and the device itself
- Device Drivers: each type of controller is different, different software is needed to control each one
 - Driver has to be put into the operating system so it can run in kernel mode
- Interrupts
 - Devices generate interrupts and the operating system runs interrupt handler code
 - Mapping of interrupts to handler code stored in memory in the interrupt vector table

Figure 1-11

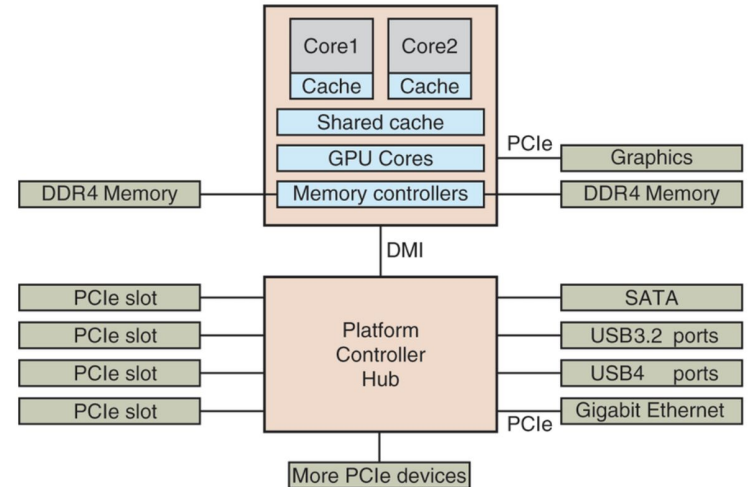


(a) The steps in starting an I/O device and getting an interrupt. (b) Interrupt processing involves taking the interrupt, running the interrupt handler, and returning to the user program.

Busses

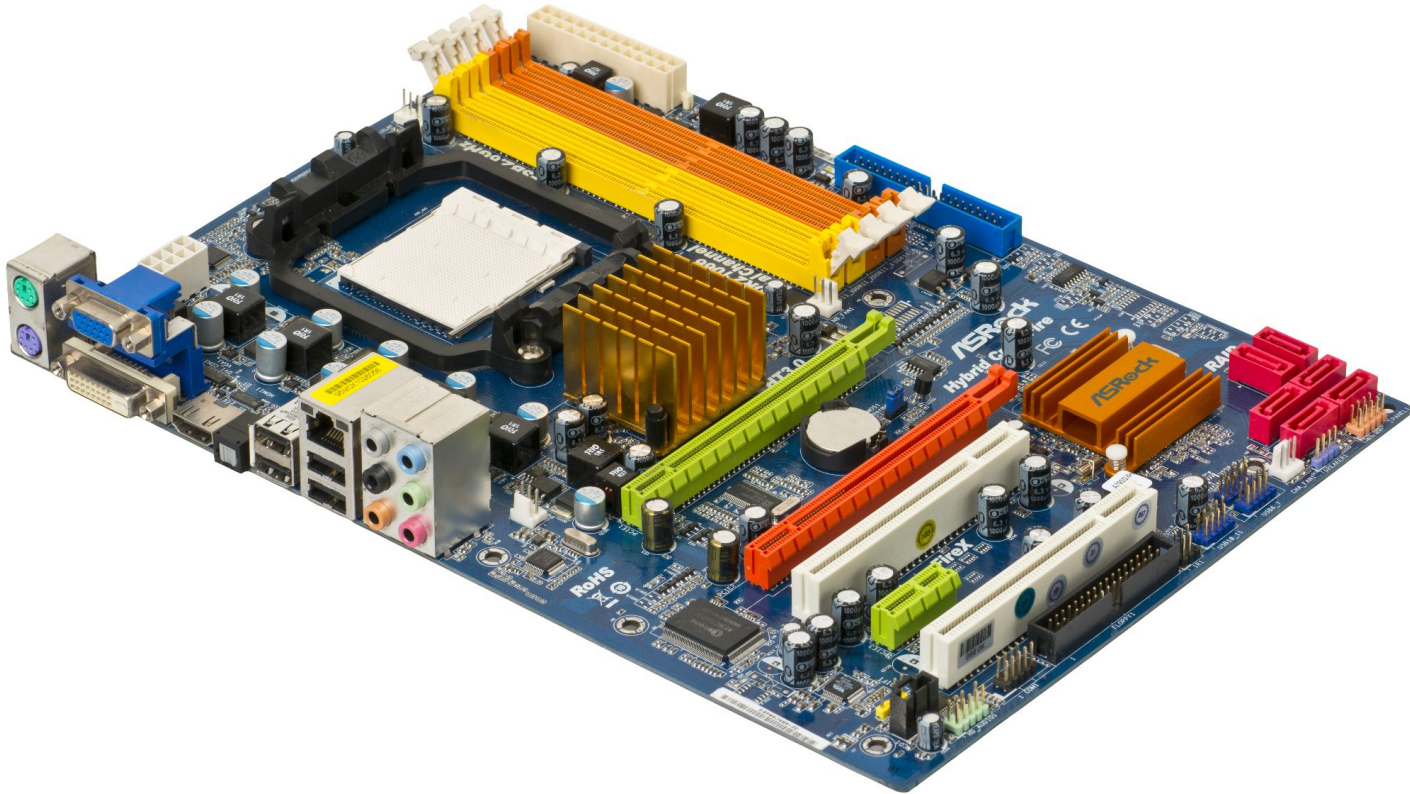
- Large x86 system includes multiple buses: cache, memory, PCIe, PCI, USB, SATA, and DMI
 - Different transfer rates and functions
- Peripheral Component Interconnect Express (PCIe)
 - Utilizes dedicated, point-to-point connections instead of shared bus architecture
 - Serial bus architecture
- CPU communicates with memory through a fast DDR4 bus, external graphics via PCIe, and other devices via a DMI bus

Figure 1-12



The structure of a large x86 system.

Motherboard

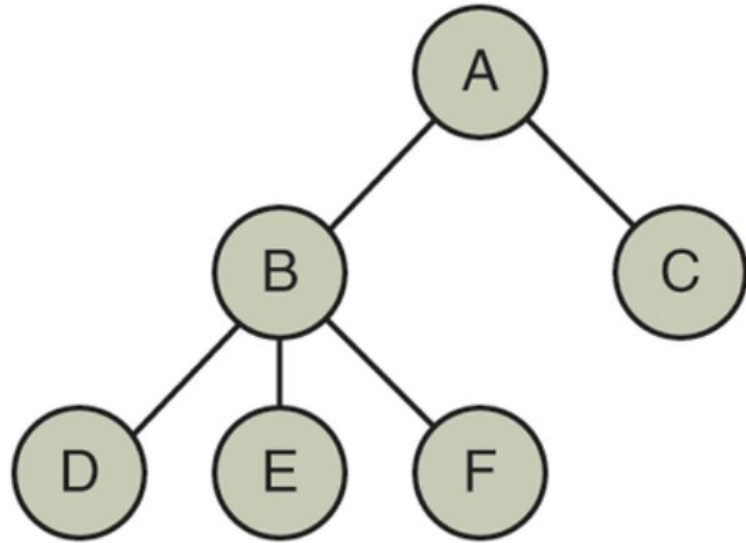


Operating System Concepts

- Processes
- Address Space
- Files
- I/O
- Protection
- Shell

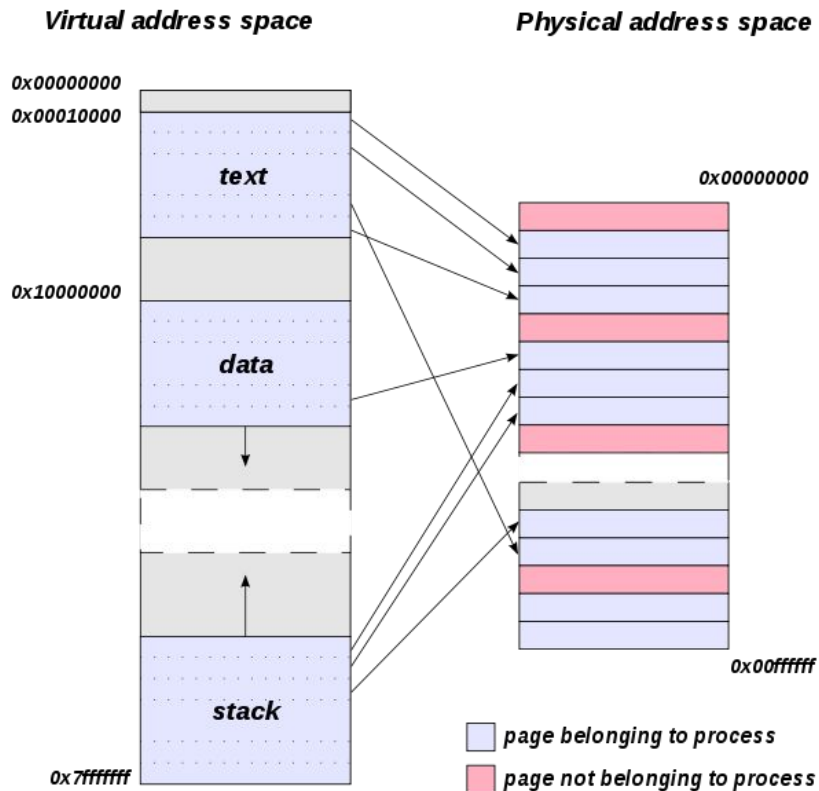
Processes

Figure 1-13



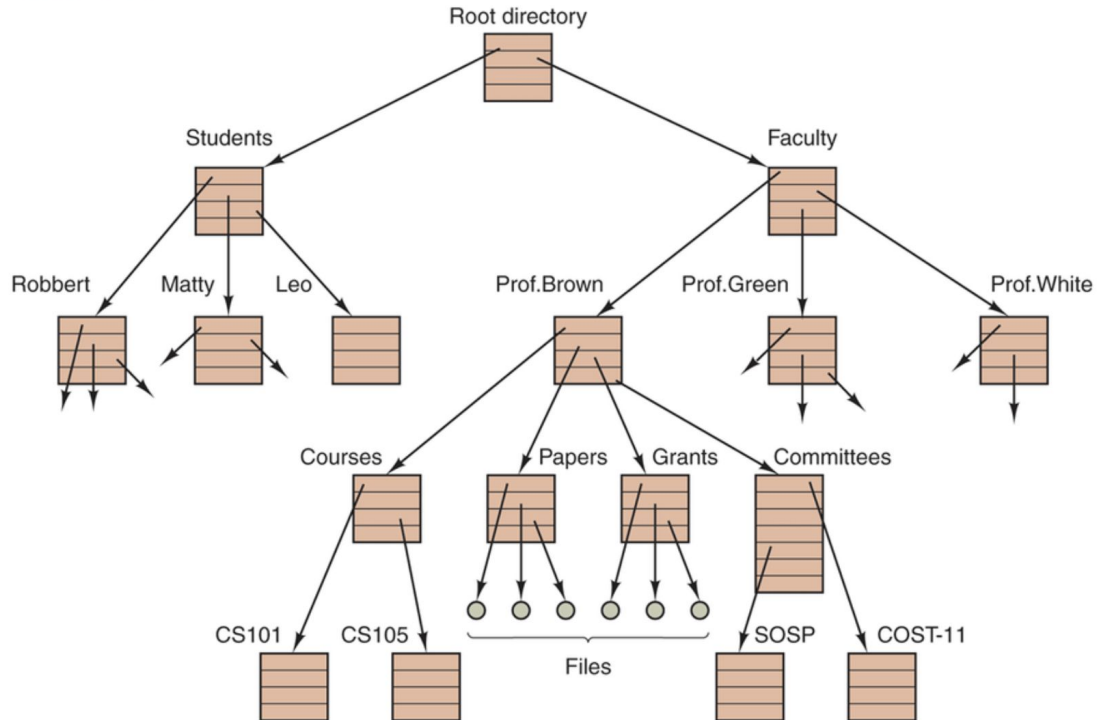
A process tree. Process A created two child processes, B and C. Process B created three child processes, D, E, and F.

Address Space



Files

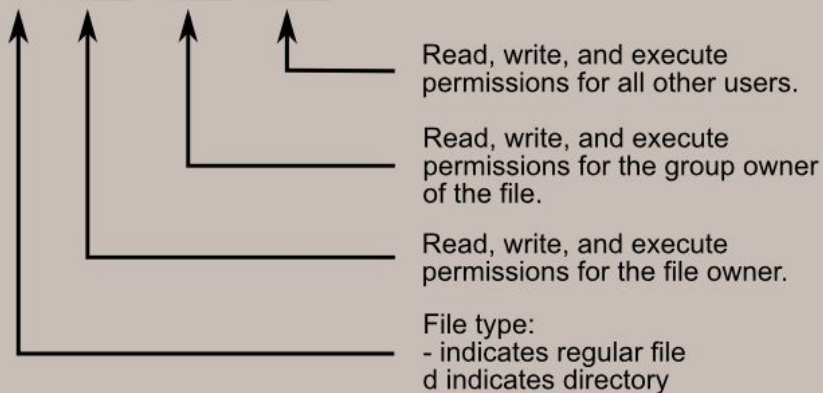
Figure 1-14



A file system for a university department.

Protection

- rwxrwxrwx



```
workstation@workstation:~/reth$ ls -la
total 356
drwxrwxr-x 12 workstation workstation 4096 Aug 14 19:36 .
drwxr-xr-x  6 workstation workstation 4096 Aug 16 11:24 ..
drwxrwxr-x  2 workstation workstation 4096 Aug 14 19:36 assets
drwxrwxr-x  3 workstation workstation 4096 Aug 14 19:36 bin
drwxrwxr-x  9 workstation workstation 4096 Aug 14 19:36 book
-rw-rw-r--  1 workstation workstation  466 Aug 14 19:36 book.toml
-rw-rw-r--  1 workstation workstation 201326 Aug 14 19:36 Cargo.lock
-rw-rw-r--  1 workstation workstation  4426 Aug 14 19:36 Cargo.toml
-rw-rw-r--  1 workstation workstation   14 Aug 14 19:36 .clippy.toml
-rw-rw-r--  1 workstation workstation  1538 Aug 14 19:36 codecov.yml
-rw-rw-r--  1 workstation workstation   842 Aug 14 19:36 CODEOWNERS
-rw-rw-r--  1 workstation workstation 11614 Aug 14 19:36 CONTRIBUTING.md
drwxrwxr-x 20 workstation workstation 4096 Aug 14 19:36 crates
-rw-rw-r--  1 workstation workstation   215 Aug 14 19:36 Cross.toml
-rw-rw-r--  1 workstation workstation  3437 Aug 14 19:36 deny.toml
-rw-rw-r--  1 workstation workstation  1059 Aug 14 19:36 Dockerfile
-rw-rw-r--  1 workstation workstation   533 Aug 14 19:36 Dockerfile.cross
```

Shell

```
trueblocks-core — -zsh — 118x45
~/Documents/Projects/trueblocks-core — -zsh

1213 ls -la
1214 cd Documents
1215 cd Projects
1216 ls
1217 cd trueblocks-core
1218 cat README.md
manning@Michaels-iMac trueblocks-core % cat README.md
<!-- markdownlint-disable MD033 MD036 MD041 -->
<b>If you have an existing installation, complete applicable [Migrations](https://github.com/TrueBlocks/trueblocks-core/blob/develop/MIGRATIONS.md) before proceeding.</b><hr>

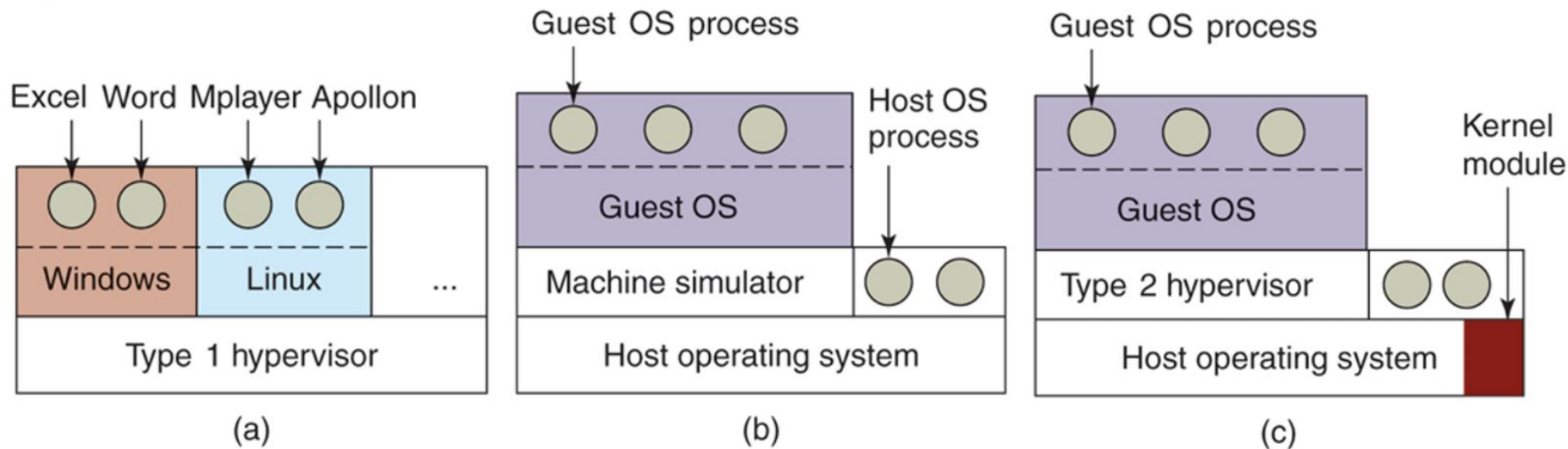
<h1>TrueBlocks / Unchained Index</h1>

![GitHub repo size](https://img.shields.io/github/repo-size/TrueBlocks/trueblocks-core)
[[GitHub contributors](https://img.shields.io/github/contributors/TrueBlocks/trueblocks-core)](https://github.com/TrueBlocks/trueblocks-core/contributors)
[[GitHub stars](https://img.shields.io/github/stars/TrueBlocks/trueblocks-core?style%3Dsocal)](https://github.com/TrueBlocks/trueblocks-core/stargazers)
[[GitHub forks](https://img.shields.io/github/forks/TrueBlocks/trueblocks-core?style=social)](https://github.com/TrueBlocks/trueblocks-core/network/members)
[[Twitter Follow](https://img.shields.io/twitter/follow/trueblocks?style=social)](https://twitter.com/trueblocks)

**Table of Contents**
```

Virtual Machines

Figure 1-29



(a) A type 1 hypervisor. (b) A pure type 2 hypervisor. (c) A practical type 2 hypervisor.

Containers

